

Proposta de Solução Tecnológica:

Project Pulse

Status do Documento: Diagnóstico & Fronteiras de Engenharia (Pronto para Revisão)

Autor: Taino Samuel Lima Ribeiro

Data de Entrega: 01 de Junho de 2026

Abordagem: Conforme o Método DIZE (Diagnosticar, Intervir, Zipar, Escalar)

1. DIAGNÓSTICO (O Entendimento da Frustração)

1.1. Contexto da Liderança e Gargalo de Operações

Em empresas de alta performance, a tomada de decisão ágil é impedida pela fragmentação de dados brutos. Lideranças e gestores executivos sofrem com o tempo gasto em microgerenciamento de tarefas em ferramentas de gestão como o ClickUp. Sem uma visão consolidada e priorizada, é impossível rastrear rapidamente demandas paradas ou em situação de risco.

- **A Frustração Principal:** Falta de um painel analítico centralizado que mostre a saúde real dos projetos e indique de forma automática quais tarefas necessitam de apoio ou intervenção urgente da gerência.
- **Impacto Operacional e de Negócio:** Atraso na entrega de marcos contratuais (*milestones*), aumento no custo operacional de acompanhamento e falha em mitigar gargalos críticos antes do vencimento das demandas.

1.2. Mapeamento de Requisitos (Fronteiras do Sistema)

Requisitos Funcionais (RF)

- **[RF01] Integração Direta ClickUp:** O sistema deve consumir tarefas reais de uma lista pré-configurada no ClickUp utilizando autenticação via Token de API Pessoal no cabeçalho.
- **[RF02] Pipeline de Processamento (Engine de Criticidade):** Antes de entregar os dados para exibição, o motor de inteligência deve injetar o campo lógico calculado `status_critico` (booleano) com base em regras de negócio de tempo e prioridade.
- **[RF03] Painel de Status Executivo:** Interface baseada em colunas padronizadas de fluxo Kanban: *To Do*, *Doing* e *Done*.
- **[RF04] Alerta Visual de Risco:** Destacar explicitamente na interface todo card que possuir `status_critico == true` (usando badges vermelhos, ícones de atenção ou bordas de destaque).

- **[RF05] Filtros de Gestor:** Sistema dinâmico de pesquisa em tela por nome da tarefa ou membro responsável designado (*assignee*).
- **[RF06] Dashboard Analítico (Diferencial de Negócio):** Exibição de cards estatísticos agregados no topo do painel mostrando o total geral de demandas, número de tarefas críticas ativas e demandas finalizadas.

Requisitos Não-Funcionais (RNF)

- **[RNF01] Segurança no Tráfego de Credenciais:** Nenhuma chave privada ou Token de API Pessoal do ClickUp deve ser exposta no navegador do cliente (frontend).
- **[RNF02] Desacoplamento Arquitetural (BFF Pattern):** O frontend (Next.js/React) atuará apenas como uma camada de exibição leve e reativa. Toda a lógica de requisição à API externa do ClickUp, parsing e injeção de dados será processada pelo servidor Node.js intermediário de forma segura.

2. REGRAS DE NEGÓCIO DO MOTOR (A Inteligência do Dado)

O motor de processamento lógico calcula o campo `status_critico` por meio de uma operação lógica de disjunção inclusiva baseada no estado da tarefa.

Seja T uma tarefa do ClickUp, definimos os parâmetros:

- $P_T \in \{"low", "normal", "high", "urgent"\}$: a prioridade da tarefa.
- t_{last} : o timestamp UTC da última atualização registrada na tarefa (derivado de `date_updated`).
- t_{now} : o timestamp UTC do momento da requisição ao servidor.
- $\Delta t = t_{now} - t_{last}$: a diferença temporal (em milissegundos) desde a última atualização da tarefa.

Definimos que o campo lógico calculado $S_{critico} \in \{true, false\}$ obedece estritamente à seguinte regra matemática:

$$S_{critico} = true \iff (P_T = "urgent" \vee \Delta t > 259.200.000 \text{ ms})$$

Onde **259.200.000 ms** corresponde exatamente à constante temporal de **3 dias** de inatividade operativa:

$$3 \text{ dias} \times 24 \text{ h} \times 60 \text{ min} \times 60 \text{ s} \times 1000 \text{ ms}$$

3. VISÃO GERAL DA ARQUITETURA (High-Level Design)

Utilizaremos uma arquitetura robusta, segura e desacoplada baseada no padrão BFF (Backend For Frontend) para orquestrar as consultas de dados de forma assíncrona.

graph TD

```
Client[Next.js Frontend] -->|1. GET /api/tasks| BFF[Node.js Express Server]
BFF -->|2. GET Authenticated Request| ClickUp[API Oficial ClickUp]
ClickUp -->|3. Retorna JSON de Tarefas Bruto| BFF
BFF -->|4. Aplica Engine de Inteligência status_critico| BFF
BFF -->|5. Retorna Payload Otimizado e Filtrado| Client
```

Justificativa de Engenharia do Modelo Node.js/Express BFF

1. **Isolamento de Segurança:** O Token de API do ClickUp fica gravado exclusivamente nas variáveis de ambiente seguras do servidor Node.js (.env), impedindo qualquer vazamento ou exposição no client-side.
2. **Performance e Consistência de Dados:** Ao delegar o tratamento de dados (parsing de datas, normalização de status e injeção do campo status_critico) ao servidor Express, o frontend Next.js lida apenas com dados limpos e prontos para renderização. Isso otimiza o uso de memória no cliente.
3. **Desacoplamento de Integração:** Mudanças nos endpoints ou nas regras de negócio da API do ClickUp podem ser absorvidas inteiramente no backend, sem a necessidade de novos deploys de interface do frontend.

4. ZIPAR (Estratégia de MVP e Desenvolvimento Ágil)

4.1. Escopo de Entrega do MVP (Minimizando Complexidade)

A Dizevolv preza pelo "cuidado na entrega". O plano de entrega foi dimensionado para produzir o máximo impacto estratégico com o menor overhead de engenharia possível.

- **Fase 1 (Core/MVP - Entrega em 24h):**
 - Desenvolvimento do servidor Express independente conectando com a API real do ClickUp.
 - Implementação da Engine de Criticidade no servidor.
 - Interface Kanban responsiva dividida em 3 colunas (To Do, Doing, Done) e o dashboard analítico no topo.
- **Fase 2 (Escala e Refinamento - Próximas Sprints):**
 - Implementação de cache local no backend (Redis ou NodeCache) para respeitar o limite de Rate Limiting da API do ClickUp.
 - Sistema de autenticação SSO para gestores e relatórios analíticos em PDF.

4.2. Metodologia de Desenvolvimento (Workflow Transparente)

1. **Versionamento:** Commits semânticos no Git documentando a evolução do projeto passo a passo.
2. **Setup Rápido:** Arquivos estruturados de inicialização que permitem a qualquer desenvolvedor rodar as duas aplicações em menos de 3 passos utilizando comandos simples de terminal.

5. ESCALAR (Transparência, Riscos e Trade-offs)

Como uma assessoria de alta performance, apresentamos as escolhas complexas de design feitas para este produto:

5.1. Análise de Trade-offs (As Escolhas Difíceis)

- **Abordagem Escolhida:** Utilização de duas aplicações separadas (Frontend Next.js e Backend Express) em vez de um monólito Next.js com API Routes integradas.
 - *Prós:* Separação completa de responsabilidades. Se o tráfego na interface crescer absurdamente, podemos escalar as instâncias de renderização do Next.js de forma isolada, enquanto a API em Node.js mantém um fluxo de requisições estável com o ClickUp.
 - *Contras:* Aumento sutil no gerenciamento de deploy (dois servidores para subir), que foi mitigado com documentação clara no README.md.

5.2. Gestão de Limitações e Mitigação de Riscos

A API oficial do ClickUp possui limites estritos de requisições por minuto (*Rate Limit*). Em uma escala real, múltiplos usuários recarregando o dashboard simultaneamente estourariam essa cota.

- *Mitigação Proposta:* Implementar uma camada de cache temporário (TTL de 5 minutos) no servidor Express. Se duas requisições de dashboard ocorrerem próximas, a segunda será respondida a partir da memória do backend, sem atingir a API do ClickUp de novo.